

Programming in Python

Maulana Hafidz Ismail

31 Oct 2025

Contents

1	Memulai dengan Python	1
1.1	Tentang Python	1
1.2	Sejarah Pemrograman	1
1.3	Dasar-dasar Pemrograman	1
1.4	Apa itu Pemrograman?	1
1.5	Manfaat Belajar Python	2
1.6	Aplikasi Python	2
1.7	Menjalankan Python di Command Line	2
1.8	Sintaksis Python dan Pentingnya Spasi	2
1.9	Memberi Komentar pada Kode	3
1.9.1	Single-line comments	4
1.9.2	Multi-line comments	4
1.9.3	Inline comments	4
1.10	Pengertian Variabel	4
1.11	Penamaan Variabel	4
1.12	Mengubah dan Menghapus Variabel	5
1.13	Deklarasi Beberapa Variabel	5
1.14	Tipe Data dalam Python	5
1.15	Jenis Tipe Data Utama	5
1.16	Menentukan Tipe Data	6
1.17	Deklarasi Strings	6
1.18	Mengakses dan Mengubah String	7
1.19	Fungsi dan Operasi pada Strings	7
1.20	Contoh Penggunaan	7
1.21	Cheatsheets Tipe Data dan Fungsi	7
1.21.1	Tipe Data	7
1.21.2	Flow Control	7
1.21.3	Indentasi	8
1.21.4	Komentar	8
1.21.5	Built-in Functions	9
1.21.6	Membuat Fungsi	10
1.22	Apa itu Typecasting?	10
1.23	Implicit Type Conversion	10
1.24	Explicit Type Conversion	10
1.25	Fungsi Typecasting Lainnya	11
1.26	Fungsi Input	11
1.27	Fungsi Print	11
1.28	Parameter Tambahan untuk Print	11
1.29	Contoh Penggunaan Input dan Print	11
1.30	Penanganan Tipe Data	12
1.31	Penggabungan String	12
1.32	Melihat Lebih Dalam ke Type Casting	12
1.33	Operator Matematika	14
1.34	Operator Logika	15

1.35	Contoh Penggunaan Operator dalam Pernyataan Kondisional . .	15
1.36	Apa itu Control Flow?	15
1.37	Conditional Statements	16
1.38	Contoh Penggunaan Control Flow	16
1.39	Penggunaan Match Statement	16
1.40	Perbandingan dengan if Statement	17
1.41	Fitur Penting dari Match Statement	17
1.42	For Loop	17
1.43	While Loop	17
1.44	Enumerate Function	18
1.45	Catatan Penting terkait Looping	18
1.46	Pengertian Nested Loops	19
1.47	Cara Kerja Nested Loops	19
1.48	Contoh Penggunaan Nested Loops	19
1.49	Visualisasi Nested Loops	19
1.50	Kompleksitas Waktu	19
2	Dasar Programming dengan Python	21
2.1	Definisi Fungsi	21
2.2	Cara Mendeklarasikan Fungsi	21
2.3	Penggunaan Fungsi	21
2.4	Contoh Praktis	21
2.5	Keuntungan Menggunakan Fungsi	21
2.6	Jenis-jenis Scope	22
2.7	Penjelasan dan Contoh	22
2.8	Apa itu Struktur Data?	23
2.8.1	Mutabilitas dan Imutabilitas	23
2.9	Pengertian List	24
2.10	Deklarasi List	24
2.11	Akses dan Indeks List	24
2.12	List Bersarang	24
2.13	Menambahkan Item ke List	24
2.14	Menghapus Item dari List	25
2.15	Iterasi Melalui List	25
2.16	Apa itu Tuple?	25
2.17	Cara Mendeklarasikan Tuple	25
2.18	Mengakses Elemen dalam Tuple	25
2.19	Menentukan Tipe Tuple	25
2.20	Metode pada Tuple	26
2.21	Perbedaan antara Tuple dan List	26
2.22	Definisi Sets	26
2.23	Operasi Dasar pada Sets	26
2.24	Operasi Matematika pada Sets	26
2.25	Catatan Penting Terkait Set	27
2.26	Apa itu Dictionary?	27
2.27	Keuntungan Menggunakan Dictionary	27

2.28	Cara Mengakses dan Memanipulasi Dictionary	27
2.29	Iterasi Melalui Dictionary	28
2.30	Contoh Pembuatan dan Penggunaan Dictionary	28
2.31	Apa itu args?	28
2.32	Apa itu kwargs?	29
2.33	Struktur data mana yang harus dipilih?	29
2.33.1	Contoh: Daftar Pegawai	29
2.34	Errors	30
2.35	Exceptions	31
2.36	Penanganan Exceptions	31
2.37	Penanganan Exceptions	31
2.38	Mengakses Informasi Exception	32
2.39	Menangani Beberapa Exceptions	32
2.40	File Handling: open dan close	32
2.41	With Statement	32
2.42	Membuat File	33
2.43	Menyisipkan Konten	33
2.44	Mengganti dan Menambahkan Konten	33
2.45	Menangani Kesalahan (Exception)	33
2.46	Metode untuk Membaca File	34
2.47	Jalur File	34
2.48	Contoh Penggunaan Open Statement	35
3	Paradigma Pemrograman	36
4	Modul, Paket, Pustaka, and Alat-alat	37

1 Memulai dengan Python

1.1 Tentang Python

Python dinamai setelah acara TV "Monty Python's Flying Circus" dan bukan dari ular piton. Nama ini dipilih karena dianggap singkat dan misterius. Python digunakan dalam aplikasi yang sering kita gunakan, seperti Instagram, Facebook, Google, dan Spotify. Contoh penggunaan Python dalam machine learning termasuk TensorFlow, yang digunakan oleh Airbnb untuk mengklasifikasikan gambar dan oleh perusahaan kesehatan untuk mengklasifikasikan data MRI. Python dianggap mudah digunakan dan sederhana, dengan banyak pustaka (libraries) yang mendukung pengembangan. Memungkinkan pengembang untuk membuat fitur dengan cepat dan melihat hasilnya lebih cepat.

1.2 Sejarah Pemrograman

- Charles Babbage, pada tahun 1822, menciptakan difference engine untuk mengurangi kesalahan manusia dalam perhitungan.
- Babbage juga mengembangkan analytical engine, yang dianggap sebagai dasar dari komputer modern.
- Teman Babbage, Ada Lovelace, menulis dokumen yang menjelaskan bagaimana analytical engine dapat melakukan urutan perhitungan, mirip dengan apa yang dilakukan oleh program komputer.

1.3 Dasar-dasar Pemrograman

- Komputer hanya memahami binary code, yang terdiri dari dua digit: 0 dan 1. Binary Code: Sistem bilangan yang hanya menggunakan dua angka, 0 (off) dan 1 (on).
- Setiap program yang ditulis harus dikonversi menjadi machine code (kode mesin) agar dapat dipahami oleh komputer. Contoh konversi: Decimal 1 menjadi Binary 1, Decimal 2 menjadi Binary 10.
- Komputer menggunakan transistors untuk merepresentasikan binary code. Transistor: Komponen elektronik yang berfungsi sebagai saklar untuk mengontrol aliran listrik.

1.4 Apa itu Pemrograman?

Pemrograman adalah kemampuan untuk memberikan serangkaian instruksi kepada komputer dalam bahasa yang dapat dipahami.

Pemrograman adalah keterampilan yang dapat diasah dengan latihan. Semakin sering berlatih, semakin baik kemampuan pemrograman seseorang.

Pemrograman juga merupakan keterampilan kreatif, karena ada banyak cara untuk menyelesaikan masalah dengan menulis program.

1.5 Manfaat Belajar Python

- Mudah Dipelajari: Python memiliki sintaks yang mirip dengan bahasa Inggris, sehingga lebih mudah dibaca dan dipahami.
- Produktivitas Tinggi: Dengan Python, pengembang dapat menyelesaikan proyek lebih cepat karena sintaks yang sederhana dan memerlukan lebih sedikit kode dibandingkan dengan bahasa lain seperti C atau Java.

1.6 Aplikasi Python

- Berbagai Platform: Python dapat digunakan di Windows, Mac, dan Linux.
- Bidang Penggunaan: Python banyak digunakan dalam pengembangan web, kecerdasan buatan, machine learning, analisis data, dan aplikasi pemrograman lainnya.

1.7 Menjalankan Python di Command Line

- Python Shell:

Ini adalah cara pertama untuk menjalankan program Python. Shell ini berguna untuk menjalankan dan menguji skrip kecil tanpa perlu membuat file baru dengan ekstensi .PY. Anda dapat mengetikkan potongan kode langsung di shell.

Cukup ketik `python` di command line untuk membuka Python shell, lalu masukkan kode yang ingin dijalankan.

- Menjalankan File Python

Cara kedua adalah menjalankan file Python langsung dari command line. Setiap file dengan ekstensi .PY dapat dijalankan dengan perintah tertentu.

Ketik `python nama_file.py` di command line untuk menjalankan file tersebut.

1.8 Sintaksis Python dan Pentingnya Spasi

Menggunakan `print("Hello")` untuk menghasilkan teks. `print` adalah fungsi yang digunakan untuk menampilkan output ke layar.

```
print("Hello") print("World")
```

Jika mencoba menulis dua pernyataan `print` dalam satu baris tanpa pemisah, akan muncul error `SyntaxError: invalid syntax`. `SyntaxError` adalah kesalahan yang terjadi ketika kode tidak mengikuti aturan sintaks yang benar.

Mengatasi Error:

- Metode Pertama: Memindahkan pernyataan `print` kedua ke baris baru.

```
print("Hello")
print("World")
```

- Metode Kedua: Memisahkan pernyataan dengan titik koma.

```
print("Hello"); print("World")
```

Whitespace adalah karakter kosong (spasi, tab, newline) yang tidak terlihat tetapi mempengaruhi struktur kode. Menambahkan spasi di sekitar operator tidak menyebabkan masalah, tetapi menambahkan baris baru bisa menyebabkan error. Contohnya seperti ini:

```
#any ammount of whitespace on a single line is ok
x      =      1      +      2
```

Dan berikut tidak benar:

```
x = 1
+ 2
```

Indentasi adalah penggunaan spasi atau tab untuk mengatur blok kode, yang menunjukkan hierarki dan struktur. Penting untuk struktur kontrol seperti if statement.

```
if name == "John":
    print(name)
```

Jika indentasi dihapus, akan muncul error `IndentationError: expected an indented block`.

1.9 Memberi Komentar pada Kode

Menambahkan komentar ke kode tidak hanya membantu Anda sebagai pengembang, tetapi juga membantu anggota tim lainnya. Komentar sangat berguna untuk menyegarkan ingatan Anda tentang kode yang mungkin telah Anda tulis berbulan-bulan lalu dan Anda mungkin lupa apa tujuannya. Ada beberapa alasan mengapa Anda perlu menambahkan komentar ke berkas kode. Alasannya antara lain:

- Menjelaskan tujuan kode.
- Memberitahu pengembang bahwa kode tersebut sudah usang.
- Menambahkan komentar TODO untuk pekerjaan yang akan diselesaikan nanti.

Berikut adalah contoh berbagai jenis komentar yang dapat digunakan.

1.9.1 Single-line comments

Penggunaan simbol `#` memberi tahu Python untuk mengabaikan semua yang ada setelah titik ini hingga akhir baris saat ini.

```
# Don't try to Run Me, I'm a comment  
x = 10
```

1.9.2 Multi-line comments

Tanda Kutip Tiga (''' atau '''): Python menggunakan tanda kutip tunggal atau ganda tiga kali untuk membuat komentar multi-baris. Tanda kutip ini biasanya digunakan untuk docstring atau komentar yang mencakup beberapa baris.

1.9.3 Inline comments

Simbol `#` memberi tahu Python untuk mengabaikan semua yang ada setelah titik ini hingga akhir baris saat ini. Hal ini memungkinkan komentar ditempatkan di dalam kode itu sendiri. Komentar sebaris sebaiknya digunakan untuk memberikan informasi penting tentang kode yang ditanganinya.

```
x = 1 # Resetting buffer size
```

Ingatlah untuk selalu memiliki alasan untuk berkomentar, komentar harus memberikan informasi kepada pembaca dan tidak mengganggu.

1.10 Pengertian Variabel

Variabel adalah tempat untuk menyimpan data yang dapat berubah. Dalam Python, Anda dapat mendeklarasikan variabel dengan memberikan nama dan nilai.

Contoh sintaks: `X = 10` (di mana `X` adalah nama variabel dan `10` adalah nilainya).

1.11 Penamaan Variabel

Penting untuk memberikan nama yang bermakna agar mudah dipahami oleh programmer lain.

Dua konvensi penamaan yang umum digunakan:

- **camelCase**: Huruf pertama dari kata pertama kecil, dan huruf pertama dari setiap kata berikutnya besar. Contoh: `myName`.
- **snake_case**: Semua huruf kecil dengan garis bawah sebagai pemisah. Contoh: `my_name`.

1.12 Mengubah dan Menghapus Variabel

Anda dapat mengubah nilai variabel yang sudah dideklarasikan dengan cara mendeklarasikannya kembali.

```
a = 10
print(a) # Output: 10
a = 5
print(a) # Output: 5
```

Untuk menghapus variabel, gunakan perintah del.

```
del a
print(a) # Akan menghasilkan error: NameError: name 'a' is
        not defined
```

1.13 Deklarasi Beberapa Variabel

Anda dapat mendeklarasikan beberapa variabel sekaligus.

```
a, b, c = 1, 2, 3
print(a, b, c) # Output: 1 2 3
```

1.14 Tipe Data dalam Python

Tipe data adalah atribut yang terkait dengan data yang memberi tahu sistem komputer bagaimana menginterpretasikan nilainya.

Tipe Data memastikan data dikumpulkan dalam format yang diinginkan dan nilai setiap properti sesuai harapan.

1.15 Jenis Tipe Data Utama

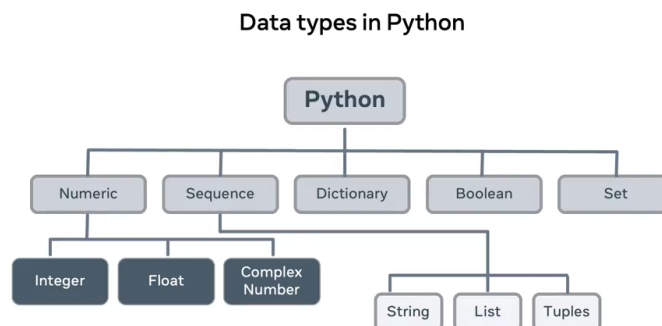


Figure 1: Pohon Tipe Data

- Numeric: Tipe data ini mencakup:

- Integer: Angka bulat tanpa desimal, misalnya 10 atau -10.
- Float: Angka dengan desimal, misalnya 10.5 atau 6.7.
- Complex: Angka kompleks yang terdiri dari bagian nyata dan imajiner, misalnya $a = 10 + 10j$.
- Sequence: Tipe data yang berisi satu atau lebih tipe dalam urutan yang teratur, termasuk:
 - String: Sekuens karakter yang diapit oleh tanda kutip, misalnya "Hello".
 - List: Sekuens yang dapat berisi berbagai tipe, diapit oleh tanda kurung siku, misalnya [1, 2, 3].
 - Tuple: Mirip dengan list, tetapi tidak dapat diubah (immutable), diapit oleh tanda kurung, misalnya (1, 2, 3).
- Dictionary: Menyimpan data dalam struktur objek key-value. Contoh:

```
ed = {'a': 22, 'b': 44.4}
```

- Boolean: Tipe data yang hanya memiliki dua nilai, yaitu True atau False. Contoh:

```
is_valid = True
```

- Set: Koleksi tidak teratur dari nilai yang tidak diulang. Contoh:

```
example_set = {1, 2, 3, 4}
```

1.16 Menentukan Tipe Data

Fungsi `type()`: Digunakan untuk menentukan tipe data dari variabel. Contoh:

```
a = 10
print(type(a)) # Output: <class 'int'>
```

1.17 Deklarasi Strings

String adalah sekuens karakter yang dikelilingi oleh tanda kutip tunggal (') atau ganda ("). Contoh sintaks: `a = 'hello'` atau `b = "hello"`.

Encoding adalah proses mengubah karakter menjadi bentuk yang dapat dipahami komputer. Python menggunakan `Unicode` untuk encoding.

1.18 Mengakses dan Mengubah String

- Indexing: Setiap karakter dalam string dapat diakses menggunakan indeks. Python menggunakan zero-indexing, artinya indeks dimulai dari 0. Contoh: `name[0]` mengakses karakter pertama.
- Mengubah Nilai: Anda dapat mengubah nilai string dengan cara menetapkan nilai baru ke variabel. Contoh: `name = 'Paul'`.

1.19 Fungsi dan Operasi pada Strings

- `len()`: Fungsi untuk memeriksa panjang string. Contoh sintaks: `len(name)` akan mengembalikan jumlah karakter dalam string.
- Concatenation: Menggabungkan dua string menggunakan operator `+`. Contoh: `a + b` akan menggabungkan string `a` dan `b`.

1.20 Contoh Penggunaan

- Single Line String: `a = 'this is a single line'`
- Multi-Line String: Untuk membuat string yang panjang, gunakan backslash (`\`) di akhir baris. Contoh:

```
b = 'this_is_a_multi-line_string_example_\nthat_continues_here.'
```

- Output: Menggunakan `print()` untuk menampilkan string. Contoh: `print(a)` akan mencetak nilai dari `a`.

1.21 Cheatsheets Tipe Data dan Fungsi

Berikut ini adalah panduan singkat tentang jenis data dalam Python.

1.21.1 Tipe Data

Tipe Data	Meaning	Example
string	Text	'Hello', "Testing 123"
integer	Numbers	-5, 4, 3, 2, 0
float	Decimals	2.4, 5.2, 1000.00

1.21.2 Flow Control

Operator Perbandingan

Operator	Meaning	Example
==	Equals	a == b
!=	Not Equal	a != b
<	Less than	a < b
>	Greater than	a > b
<=	Less than or Equal to	a <= b
>=	Greater than or Equal to	a >= b

1.21.3 Indentasi

Indentasi digunakan untuk mendefinisikan blok kode dalam Python. Berbeda dengan banyak bahasa pemrograman yang menggunakan kurung kurawal untuk mengelompokkan kode, Python menggunakan indentasi.

Indentasi wajib digunakan dalam Python dan digunakan untuk menunjukkan pernyataan mana yang termasuk dalam loop, fungsi, kondisional, dan struktur lainnya.

```
if x > 0: # Colon indicates the start of the block
    print("Positive") # Indented
else:
    print("Non-positive") # Indented
```

1.21.4 Komentar

Menempatkan simbol # di depan teks yang ingin Anda jadikan komentar akan membuat Python mengabaikan semua teks dari titik tersebut hingga akhir baris saat ini.

Triple Quotes (''' or """): Python menggunakan tanda kutip tunggal atau ganda bertiga untuk membuat komentar **multi-baris**. Tanda kutip ini umumnya digunakan untuk docstrings atau komentar yang mencakup beberapa baris.

Simbol # akan membuat Python mengabaikan semua teks mulai dari titik tersebut hingga akhir baris saat ini, sehingga komentar inline dapat dibuat dengan cara ini.

Kolon (:): Digunakan untuk menandai awal blok kode, seperti dalam perulangan, kondisi, dan definisi fungsi.

```
# Single Line comment

# This is a multiline comment
# which can be used for long comments

x = 1 # assigns value of 1 to x

if x > 0: # Colon starts the block
    print("Positive")
```

1.21.5 Built-in Functions

- `print()`

Fungsi ini mencari perangkat output default, terminal Anda, dan menampilkan nilai yang diteruskan kepadanya.

```
print("Hello")
```

- `input()`

Fungsi ini mencari perangkat input default, yaitu keyboard Anda, dan menangkap nilainya. Nilai ini kemudian dapat ditugaskan atau digunakan.

```
print("Where do you live?")  
location = input()  
print("So you live in " + location)
```

Catatan: Blok kode di atas akan menampilkan prompt kepada pengguna dengan pertanyaan "Di mana Anda tinggal?". Pengguna kemudian akan memasukkan respons dan menekan tombol Return untuk melanjutkan eksekusi kode yang tersisa. Respons yang dimasukkan oleh pengguna disimpan, dikembalikan sebagai output, atau dapat ditugaskan ke variabel tertentu tergantung pada konteks penggunaannya.

- `len()`

Fungsi ini mengembalikan panjang atau jumlah elemen yang terdapat dalam struktur yang diterapkan padanya. Struktur tersebut dapat berupa string, array, daftar, tuple, kamus, atau urutan apa pun.

```
len("Hello")  
5
```

- `str()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi String.

```
str(55)  
'55'
```

- `int()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi bilangan bulat.

```
int('75')  
75
```

- `float()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi bilangan float.

```
some_int = 10
float(some_int)
10.0
```

1.21.6 Membuat Fungsi

Fungsi dalam Python memerlukan kata kunci untuk mendefinisikannya: ‘def’ diikuti oleh identifikasi (nama), yang membentuk tanda tangan fungsi. Tubuh fungsi berisi kode yang akan dijalankan saat fungsi dipanggil.

```
def say_hello():
    return "Hello there!"

# With parameters
def say_hello(you):
    return "Hello " + you
```

1.22 Apa itu Typecasting?

Typecasting adalah proses mengubah satu tipe data menjadi tipe data lain. Dalam Python, ada dua jenis konversi: implicit dan explicit.

1.23 Implicit Type Conversion

Implicit Conversion : Dilakukan secara otomatis oleh compiler Python untuk mencegah kehilangan data. Contoh: mengubah int menjadi float jika nilai yang dimasukkan adalah desimal.

Penting untuk dicatat bahwa Python hanya dapat mengonversi nilai jika tipe data tersebut kompatibel. Misalnya, int dan float kompatibel, tetapi String dan int tidak.

1.24 Explicit Type Conversion

Explicit Conversion : Dilakukan oleh pengembang menggunakan fungsi yang disediakan oleh Python. Beberapa

- `str()`: Mengubah tipe data menjadi string. Sintaks: `str(value)`.
- `int()`: Mengubah tipe data menjadi integer. Sintaks: `int(value)`.
- `float()`: Mengubah tipe data menjadi float. Sintaks: `float(value)`.

1.25 Fungsi Typecasting Lainnya

- `ord()`: Mengembalikan integer yang mewakili karakter unicode yang mendasarinya.
- `hex()`: Mengonversi integer menjadi string heksadesimal.
- `oct()`: Mengambil integer dan mengembalikan string yang mewakili angka oktal.
- `tuple()`, `set()`, `list()`, `dict()`: Fungsi lain untuk mengonversi tipe data.

1.26 Fungsi Input

Fungsi `input()` digunakan untuk mendapatkan data dari pengguna.

Sintaks: `input("Prompt")`

- Contoh: `email = input("Please enter your email address: ")`
- Penjelasan: Pengguna akan diminta untuk memasukkan alamat email, yang kemudian disimpan dalam variabel `email`.

1.27 Fungsi Print

Fungsi `print()` digunakan untuk menampilkan output ke layar.

Sintaks: `print(value1, value2, ..., sep=' ', end='\n')`

- Contoh: `print("Hello", "World", sep=" ")`
- Penjelasan: Menampilkan "Hello, World" dengan koma sebagai pemisah.

1.28 Parameter Tambahan untuk Print

- **sep**: Menentukan pemisah antara objek yang dicetak.
- **end**: Menentukan apa yang dicetak di akhir output (default adalah new-line).
- **file**: Menentukan di mana nilai dicetak (default adalah STD out).
- **flush**: Boolean untuk mengatur apakah buffer harus dibersihkan.

1.29 Contoh Penggunaan Input dan Print

Mengambil dua angka dari pengguna dan mencetak hasil penjumlahannya:

```
num1 = int(input("Please enter the first number: "))
num2 = int(input("Please enter the second number: "))
print("The sum is:", num1 + num2)
```

Penjelasan: Menggunakan `int()` untuk mengonversi input menjadi integer sebelum melakukan penjumlahan.

1.30 Penanganan Tipe Data

Input dari fungsi `input()` selalu berupa string.

Sintaks: `type(variable)`

- Contoh: `print(type(num1))`
- Penjelasan: Menampilkan tipe data dari variabel `num1`, yang akan menunjukkan bahwa itu adalah string jika tidak dikonversi.

1.31 Penggabungan String

Menggabungkan dua atau lebih string menggunakan operator `+`.

Contoh:

```
first_name = input("Please enter your first name: ")
last_name = input("Please enter your last name: ")
print("Hello, " + first_name + " " + last_name)
```

Penjelasan: Menampilkan sapaan dengan nama depan dan belakang yang dimasukkan pengguna.

1.32 Melihat Lebih Dalam ke Type Casting

Ada beberapa situasi di mana tipe data dari suatu nilai perlu diubah menjadi tipe data lain.

Proses ini dikenal sebagai casting tipe.

Cara lain yang lebih informal untuk menyebutnya adalah "konversi tipe data".

Contoh paling sederhana dari konversi data dapat dilihat pada perbandingan berikut:

```
print(10 == 10)
```

Dalam potongan kode di atas, saya bertanya kepada Python apakah angka 10 sama dengan angka 10, dan saya akan mendapatkan nilai boolean `True` yang dicetak, yang mengonfirmasi bahwa memang keduanya sama.

Bagaimana jika saya melakukan ini?

```
print(10 == 10.00)
```

Sekali lagi, nilai boolean `True` ditampilkan di layar.

Sekarang, Anda mungkin keberatan bahwa, ya, 10 secara teknis tidak sama dengan 10.0 - karena, bisa dibilang, angka pertama adalah bilangan bulat, dan angka kedua adalah bilangan desimal. Anda benar - meskipun ini adalah angka yang sama, mereka memiliki tipe data yang berbeda.

Namun, Python di sini melakukan apa yang disebut "konversi tipe implisit".

Untuk memahami bagaimana ini bekerja, saya akan sedikit mengubah contoh sebelumnya. Alih-alih meminta Python untuk membandingkan kedua angka dan mengembalikan apakah keduanya sama atau tidak, saya akan meminta Python untuk menampilkan hasil penjumlahan kedua angka tersebut.


```
print(10 + 10.0)
```

Hasil cetak adalah 20.0.

Hasil ini memungkinkan saya menyimpulkan bahwa saat Python menjalankan operasi yang melibatkan bilangan bulat dan bilangan desimal, ia secara implisit mengubah tipe bilangan bulat menjadi bilangan desimal, lalu menyelesaikan operasi tersebut.

Untuk benar-benar menekankan poin ini, saya dapat memperluas contoh sebelumnya dengan menggunakan fungsi `type()`:

```
print(type(10 + 10.0))
```

Hasilnya adalah `<class 'float'>`, jadi ini mengonfirmasi kesimpulan saya.

Sekarang saya akan menunjukkan kepada Anda program kecil dalam Python yang bekerja dengan angka:

```
user_num_1 = input('First number is: ')
user_num_2 = input('Second number is: ')
user_sum = user_num_1 + user_num_2
print(user_sum)
```

Ketika saya menjalankan program ini, misalnya, saya dapat memasukkan nilai angka pertama sebagai 5, dan nilai angka kedua juga sebagai 5, dengan harapan nilai `user_sum` yang dicetak akan menjadi 10.

Namun, ketika saya melakukan hal itu, angka 55 yang dicetak.

Mengapa ini tidak berfungsi?

Jawabannya cukup sederhana: segala sesuatu yang diketik oleh pengguna oleh Python dikonversi menjadi tipe data string.

Ini berarti, meskipun saya mengetik angka ke dalam dua input tersebut, yang disimpan dalam `user_num_1` dan `user_num_2` sebenarnya adalah string.

Secara efektif, ini sama saja dengan jika saya hanya melakukan ini:

```
user_num_1 = "5"
user_num_2 = "5"
user_sum = user_num_1 + user_num_2
print(user_sum)
```

Kali ini, hasil cetak dari `user_sum` masih "55", tetapi hal ini tidak melanjutkan.

Untuk membuat kode Python saya berfungsi sesuai yang saya inginkan, saya perlu melakukan konversi tipe secara eksplisit.

Dengan kata lain, saya harus mengubah nilai "5" menjadi nilai 5.

Berikut adalah kode saya yang telah diperbarui:

```
user_num_1 = input('First number is: ')
user_num_2 = input('Second number is: ')
user_sum = float(user_num_1) + float(user_num_2)
print(user_sum)
```

Apa yang saya lakukan di sini adalah memastikan bahwa program saya dapat menangani input berupa bilangan float dan tetap berfungsi dengan benar.

Dengan kata lain, saya memastikan bahwa jika pengguna memasukkan nilai float 5.5 sebagai angka pertama dan kedua saat menjalankan kode di atas, outputnya tidak akan menimbulkan kesalahan. Sebaliknya, outputnya akan menjadi 11.0 seperti yang diharapkan.

Bagaimana jika saya memutuskan untuk menampilkan beberapa kata kepada pengguna, memberitahu mereka apa yang terjadi?

Ini adalah upaya untuk melakukannya:

```
num_1 = input('First number is: ')
num_2 = input('Second number is: ')
user_sum = float(num_1) + float(num_2)
print("The sum of " + num_1 + " and " + num_2 + " is " +
      user_sum)
```

Jika saya menjalankan kode di atas, akan muncul kesalahan berikut:

```
Traceback (most recent call last):
File "<string>", line 4, in <module>
TypeError: hanya dapat menggabungkan str (bukan "float")
dengan str
```

Artinya, saya tidak dapat menggabungkan string dan float seperti itu. Dengan kata lain, meskipun konversi tipe implisit Python berfungsi saat menggunakan operator + pada string dan integer, hal ini tidak berlaku untuk string dan float kecuali Anda secara eksplisit mengonversi float menjadi string.

Perlu diperhatikan juga, Anda juga tidak dapat menggabungkan string dan integer. Hal ini terjadi karena Python menerapkan keamanan tipe, artinya ia memerlukan instruksi eksplisit tentang cara menggabungkan tipe data yang berbeda dalam kasus seperti string dengan integer.

Solusi untuk ini adalah melakukan konversi tipe secara eksplisit, sebagai berikut.

```
n1 = input('First number is: ')
n2 = input('Second number is: ')
user_sum = float(n1) + float(n2)
print("The sum of " + n1 + " and " + n2 + " is " + str(
      user_sum))
```

Kali ini, hasilnya adalah: Jumlah dari 5,5 dan 5,5 adalah 11,0.

1.33 Operator Matematika

- Addition (+): Digunakan untuk menjumlahkan angka. Contoh sintaks: 2 + 3 menghasilkan 5.
- Subtraction (-): Digunakan untuk mengurangi angka. Contoh sintaks: 3 - 2 menghasilkan 1.

- Division (/): Digunakan untuk membagi angka. Contoh sintaks: `35 / 5` menghasilkan 7.0.
- Multiplication (*): Digunakan untuk mengalikan angka. Contoh sintaks: `7 * 4` menghasilkan 28.

1.34 Operator Logika

- AND: Memeriksa apakah semua kondisi benar. Contoh: `a < 5 and a < 10` hanya benar jika a lebih besar dari 5 dan kurang dari 10.
- OR: Memeriksa apakah setidaknya satu kondisi benar. Contoh: `a < 5 or b < 10` benar jika salah satu dari a atau b memenuhi kondisi.
- NOT: Mengembalikan nilai false jika hasilnya true. Contoh: `not (a < 5)` akan benar jika a tidak lebih besar dari 5.

1.35 Contoh Penggunaan Operator dalam Pernyataan Kondisional

Operator biasanya digunakan dengan pernyataan kondisional untuk mengontrol alur program. Contoh: Untuk memberikan diskon di restoran, Anda bisa memeriksa apakah pelanggan adalah anggota program loyalitas dan telah menghabiskan lebih dari \$100.

```
if in_loyalty_program and total_bill > 100:
    total_bill = total_bill - (total_bill * 0.1)
```

Contoh Lain:

- Mathematical Operators:
 - `print(2 + 2)` akan menampilkan 4.
 - `print(35 / 5)` akan menampilkan 7.0.
- Logical Operators:
 - `a = True; b = True; if a and b: print("all true")` hanya akan mencetak "all true" jika kedua a dan b adalah true.

1.36 Apa itu Control Flow?

Control Flow merupakan urutan di mana instruksi dalam program dieksekusi. Program akan mengambil tindakan yang berbeda berdasarkan keputusan yang dibuat.

Dalam Python, ada dua jenis control flow:

- Conditional Statements: Seperti `if`, `else`, dan `elif`.
- Loops: Seperti `for` loop dan `while` loop.

1.37 Conditional Statements

- if: Menyatakan bahwa jika kondisi terpenuhi, maka fungsi akan dijalankan.

```
if condition:
    # code to execute
```

- else: Menjalankan fungsi ketika semua kondisi tidak terpenuhi.

```
else:
    # code to execute if the condition is false
```

- elif: Menyatakan bahwa jika kondisi sebelumnya tidak terpenuhi, coba kondisi ini.

```
elif another_condition:
    # code to execute if this condition is true
```

1.38 Contoh Penggunaan Control Flow

Dalam contoh restoran, jika total tagihan (`bill_total`) lebih dari 100, maka diskon diterapkan.

```
bill_total = 114
discount1 = 10
if bill_total > 100:
    bill_total = bill_total - discount1
print("Total bill:", str(bill_total))
```

Untuk menambahkan diskon untuk tagihan di atas 200, gunakan elif,

```
elif bill_total > 200:
    discount2 = 20
    bill_total = bill_total - discount2
```

1.39 Penggunaan Match Statement

Match Statement diperkenalkan di Python versi 3.10, match statement memungkinkan penulisan kode yang lebih bersih dan mudah dibaca dibandingkan dengan if statement.

```
match variable:
    case value1:
        # aksi jika variable sama dengan value1
    case value2:
        # aksi jika variable sama dengan value2
    case _:
        # aksi default jika tidak ada yang cocok
```

1.40 Perbandingan dengan if Statement

If Statement mengharuskan penulisan banyak kondisi dengan if, elif, dan else, yang dapat membuat kode menjadi kompleks.

```
if http_status == 200:
    print("Success")
elif http_status == 404:
    print("Not Found")
else:
    print("Unknown")
```

1.41 Fitur Penting dari Match Statement

- Menggabungkan Beberapa Kondisi: Anda dapat menggunakan operator `or` untuk menggabungkan beberapa kondisi dalam match statement.
- Default Case: Menggunakan `case _` untuk menangani situasi di mana tidak ada nilai yang cocok, mirip dengan else dalam if statement.
- Fleksibilitas: Memberikan cara yang lebih bersih untuk menangani banyak kondisi dibandingkan dengan if statement.

1.42 For Loop

For loop adalah struktur kontrol yang digunakan untuk iterasi melalui sequence (seperti string atau array).

Sintaks:

```
for item in sequence:
    # kode yang akan dijalankan
```

Contoh:

```
str = "looping"
for item in str:
    print(item)
```

1.43 While Loop

While loop adalah struktur kontrol yang menjalankan blok kode selama kondisi tertentu.

Sintaks:

```
while condition:
    # kode yang akan dijalankan
```

Contoh:

```
count = 0
favorites = ["cake", "ice_cream", "cookies", "brownies", "
            pie"]
while count < len(favorites):
    print("I like this dessert:", favorites[count])
    count += 1
```

1.44 Enumerate Function

Fungsi `enumerate()` digunakan dalam for loop untuk mendapatkan indeks dan nilai item dalam urutan.

Sintaks:

```
for index, item in enumerate(sequence):
    # kode yang akan dijalankan
```

Menggunakan `enumerate()` untuk mencetak indeks dan nilai:

```
for idx, item in enumerate(favorites):
    print(idx, item)
```

Ini akan mencetak indeks dan nama dessert dari array `favorites`.

1.45 Catatan Penting terkait Looping

- Infinite Loop: Jika kondisi dalam while loop tidak pernah menjadi false, loop akan terus berjalan tanpa henti. Pastikan untuk mengincrement counter agar tidak terjadi infinite loop.
- Indexing: Dalam Python, indeks dimulai dari 0, jadi item pertama memiliki indeks 0 dan item terakhir memiliki indeks n-1, di mana n adalah jumlah item dalam urutan.
- Pernyataan **break** digunakan untuk menghentikan loop, yang pada gilirannya juga menghentikan kondisi else.
- Sama seperti perintah break, perintah **continue** dapat digunakan untuk mengontrol iterasi loop. Perbedaan utamanya adalah perintah continue memungkinkan Anda untuk melewati bagian tertentu dari loop, tetapi kemudian melanjutkan ke bagian sisanya.
- Pernyataan **pass** berfungsi sebagai placeholder, memungkinkan Anda untuk menyertakan blok kode kosong tanpa menyebabkan kesalahan sintaks. Pernyataan ini tidak melakukan apa-apa dan memungkinkan program untuk melanjutkan eksekusi secara normal.

1.46 Pengertian Nested Loops

Nested Loop adalah struktur pengulangan di mana satu loop (inner loop) berada di dalam loop lainnya (outer loop). Ini memungkinkan iterasi melalui beberapa koleksi data secara bersamaan.

1.47 Cara Kerja Nested Loops

- Outer Loop: Loop yang menjalankan iterasi pertama. Setelah itu, ia akan masuk ke inner loop.
- Inner Loop: Loop yang berjalan hingga batas yang ditentukan. Setelah selesai, kontrol kembali ke outer loop untuk iterasi berikutnya.

1.48 Contoh Penggunaan Nested Loops

Misalkan ada dua list dari angka satu hingga sembilan. Dengan menggunakan nested loops, kita dapat menghitung total iterasi:

```
count = 0
for i in range(1, 10): # Outer loop
    for j in range(1, 10): # Inner loop
        count += 1
print(count) # Output: 81
```

Penjelasan: Outer loop berjalan 9 kali, dan inner loop berjalan 9 kali untuk setiap iterasi outer loop, sehingga total iterasi adalah $9 * 9 = 81$.

1.49 Visualisasi Nested Loops

Untuk memahami lebih baik, kita bisa mencetak hasil dari setiap iterasi:

```
for i in range(10): # Outer loop
    for j in range(10): # Inner loop
        print(0, end=' ')
    print() # New line after inner loop
```

Penjelasan: Ini akan mencetak grid 2D dari angka nol.

1.50 Kompleksitas Waktu

Time Complexity mengacu pada waktu yang dibutuhkan untuk menjalankan kode. Semakin besar array, semakin lama waktu yang dibutuhkan.

Contoh: Jika outer loop memiliki range 100 dan inner loop 1000, waktu eksekusi akan meningkat.

```
import time
start_time = time.time()
# Code to measure
```

```
print("Execution_time:", round(time.time() - start_time, 2)  
    )
```


2 Dasar Programming dengan Python

2.1 Definisi Fungsi

Fungsi adalah sekumpulan instruksi yang dapat digunakan kembali untuk melakukan tugas tertentu. Dalam Python, fungsi dideklarasikan dengan kata kunci `def`.

2.2 Cara Mendeklarasikan Fungsi

Sintaks untuk mendeklarasikan fungsi:

```
def function_name(parameters):  
    # kode yang akan dijalankan
```

Contoh:

```
def sum(x, y):  
    return x + y
```

2.3 Penggunaan Fungsi

- Fungsi dapat menerima parameter dan mengembalikan nilai. Contoh fungsi `print()` yang digunakan untuk menampilkan output ke layar.
- Fungsi `input()` digunakan untuk menerima input dari pengguna.

2.4 Contoh Praktis

- Menghitung jumlah pajak dari total tagihan:

```
def calculate_tax(bill, tax_rate):  
    return bill * tax_rate / 100
```

- Memanggil fungsi:

```
total_tax = calculate_tax(175, 15)  
print(total_tax) # Output: 26.25
```

2.5 Keuntungan Menggunakan Fungsi

Fungsi memungkinkan kode menjadi lebih modular dan reusable. Jika ada perubahan, Anda hanya perlu memperbarui fungsi tersebut, dan semua bagian kode yang memanggil fungsi itu akan mendapatkan pembaruan.

2.6 Jenis-jenis Scope

- **Local Scope:** Variabel yang dideklarasikan dalam fungsi dan hanya dapat diakses di dalam fungsi tersebut.
- **Enclosing Scope:** Variabel yang dideklarasikan dalam fungsi luar dan dapat diakses oleh fungsi dalamnya.
- **Global Scope:** Variabel yang dideklarasikan di luar semua fungsi dan dapat diakses dari mana saja dalam kode.
- **Built-in Scope:** Variabel dan fungsi yang sudah tersedia di Python, seperti `print()` dan `len()`.

2.7 Penjelasan dan Contoh

LEGB merupakan akronim untuk Local, Enclosing, Global, dan Built-in, yang menunjukkan urutan pencarian variabel.

- Contoh Global Scope:

```
my_global = 10
def fn1():
    local_v = 5
    print("Access to global:", my_global)
fn1() # Output: Access to global: 10
print(local_v) # Akan menghasilkan error
```

Penjelasan: Variabel `my_global` dapat diakses di dalam fungsi `fn1`, tetapi `local_v` tidak dapat diakses di luar fungsi.

- Contoh Enclosing Scope:

```
def fn1():
    enclosed_v = 8
    def fn2():
        print("Access to enclosed:", enclosed_v)
    fn2()
fn1() # Output: Access to enclosed: 8
```

Penjelasan: Fungsi `fn2` dapat mengakses variabel `enclosed_v` yang dideklarasikan di dalam `fn1`.

- Built-in Scope:

Merupakan fungsi dan object yang sudah ada di Python, seperti `print()` yang dapat diakses dari semua level scope.

Namun **reserved keyword** seperti `def`, `for`, `if`, etc tidak termasuk built-in scope.

2.8 Apa itu Struktur Data?

Sejauh ini, Anda hanya menyimpan sedikit data dalam variabel. Ini adalah bilangan bulat, Boolean atau string.

Tetapi apa yang terjadi jika Anda perlu bekerja dengan informasi yang lebih kompleks, seperti kumpulan data seperti daftar orang atau daftar perusahaan?

Struktur data dirancang untuk tujuan ini.

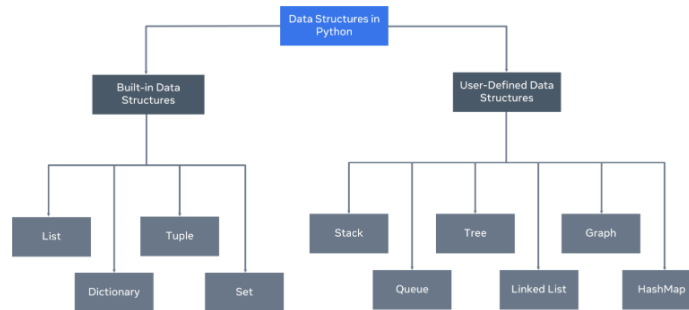


Figure 2:

Struktur data memungkinkan Anda untuk mengorganisir dan mengatur data Anda agar dapat melakukan operasi pada data tersebut. Python memiliki struktur data bawaan berikut: list, dictionary, tuple, dan set. Semua struktur data ini dianggap sebagai struktur data non-primitif, artinya mereka diklasifikasikan sebagai objek.

Selain struktur data bawaan, Python memungkinkan pengguna untuk membuat struktur data mereka sendiri. Struktur data seperti Tumpukan (Stacks), Antrian (Queues), dan Pohon (Trees) semuanya dapat dibuat oleh pengguna.

Setiap struktur data dapat dirancang untuk memecahkan masalah tertentu atau mengoptimalkan solusi yang ada agar menjadi jauh lebih efisien.

2.8.1 Mutabilitas dan Imutabilitas

Struktur data dapat bersifat mutable atau immutable. Pertanyaan selanjutnya yang mungkin Anda tanyakan adalah, apa itu mutabilitas? Mutabilitas mengacu pada data di dalam struktur data yang dapat dimodifikasi. Misalnya, Anda dapat mengubah, memperbarui, atau menghapus data saat diperlukan. List adalah contoh struktur data yang dapat diubah. Lawan dari mutabilitas adalah tidak dapat diubah. Struktur data yang tidak dapat diubah tidak akan mengizinkan modifikasi setelah data ditetapkan. Tuple adalah contoh struktur data yang tidak dapat diubah.

2.9 Pengertian List

List adalah sebuah urutan dari satu atau lebih tipe data yang berbeda atau serupa. Dalam Python, list adalah array dinamis yang dapat menyimpan tipe data apa pun.

2.10 Deklarasi List

Untuk membuat list, gunakan sintaks:

```
list1 = [1, 2, 3, 4, 5] # List berisi angka
list2 = ["A", "B", "C"] # List berisi string
list3 = ["Hello", 10, True, 3.14] # List dengan berbagai
    tipe data
```

2.11 Akses dan Indeks List

List diindeks mulai dari 0. Untuk mengakses item, gunakan sintaks:

```
list1[2] # Mengakses item ketiga, hasilnya 3
```

2.12 List Bersarang

Nested List: List yang berisi list lain. Contoh:

```
list4 = [1, [2, 3, 4], 5, 6] # List dengan nested list
```

2.13 Menambahkan Item ke List

- `insert()`: Menambahkan item pada indeks tertentu.

```
list1.insert(5, 6) # Menambahkan 6 di akhir list
```

- `append()`: Menambahkan item di akhir list

```
list1.append(6) # Menambahkan 6 di akhir list
```

- `extend()`: Menambahkan beberapa item sekaligus

```
list1.extend([6, 7, 8, 9]) # Menambahkan 6, 7, 8, 9
```

2.14 Menghapus Item dari List

- `pop()`: Menghapus item berdasarkan indeks.

```
list1.pop(4) # Menghapus item kelima
```

- `del`: Menghapus item berdasarkan indeks.

```
del list1[2] # Menghapus item ketiga
```

2.15 Iterasi Melalui List

Untuk mengakses semua item dalam list, gunakan loop:

```
for x in list1:  
    print(x) # Mencetak semua nilai dalam list
```

2.16 Apa itu Tuple?

Tuple adalah struktur data yang dapat menyimpan berbagai jenis data dalam satu variabel. Contoh: Tuple dapat berisi integer, string, double, dan boolean.

2.17 Cara Mendeklarasikan Tuple

Untuk mendeklarasikan tuple, gunakan tanda kurung:

```
my_tuple = (1, "string", 4.5, True)
```

Anda juga dapat mendeklarasikan tuple tanpa tanda kurung, tetapi penggunaan tanda kurung adalah praktik terbaik.

2.18 Mengakses Elemen dalam Tuple

Untuk mengakses elemen dalam tuple, gunakan indeks:

```
print(my_tuple[1]) # Mengakses elemen dengan indeks 1
```

Indeks dimulai dari 0, jadi `my_tuple[1]` akan mengembalikan "string".

2.19 Menentukan Tipe Tuple

Untuk mengetahui tipe dari tuple, gunakan fungsi `type()`

```
print(type(my_tuple)) # Mengembalikan <class 'tuple'>
```

2.20 Metode pada Tuple

- `count()`: Menghitung jumlah kemunculan nilai dalam tuple.

```
my_tuple.count("string") # Mengembalikan 1
```

- `index()`: Mengembalikan indeks dari nilai yang dicari.

```
my_tuple.index(4.5) # Mengembalikan 2
```

2.21 Perbedaan antara Tuple dan List

Nilai dalam tuple tidak dapat diubah setelah dideklarasikan. Jika Anda mencoba mengubah nilai, akan muncul error:

```
my_tuple[0] = 5 # Akan menghasilkan TypeError
```

2.22 Definisi Sets

Set adalah sequence yang tidak mengizinkan nilai duplikat dan tidak memiliki urutan.

2.23 Operasi Dasar pada Sets

- Menambahkan Elemen:
Sintaks: `set_a.add(6)` Definisi: Menambahkan elemen baru ke dalam set.
- Menghapus Elemen:
Sintaks: `set_a.remove(2)` atau `set_a.discard(2)` Definisi: Menghapus elemen dari set. `remove` akan menghasilkan error jika elemen tidak ada, sedangkan `discard` tidak.

2.24 Operasi Matematika pada Sets

- Union:
Sintaks: `set_a.union(set_b)` atau `set_a | set_b` Definisi: Menggabungkan dua set dan menghilangkan nilai duplikat.
- Intersection:
Sintaks: `set_a.intersection(set_b)` atau `set_a & set_b` Definisi: Mengambil elemen yang ada di kedua set.
- Difference:
Sintaks: `set_a.difference(set_b)` atau `set_a - set_b` Definisi: Mengambil elemen yang hanya ada di `set_a` dan tidak ada di `set_b`.

- Symmetrical Difference:

Sintaks: `set_a.symmetric_difference(set_b)` atau `set_a ^ set_b`

Definisi: Mengambil elemen yang ada di salah satu set tetapi tidak di kedua set.

2.25 Catatan Penting Terkait Set

- Tidak Terurut: Set tidak memiliki indeks, sehingga tidak bisa diakses dengan cara seperti `set_a[0]`. Jika dicoba, akan menghasilkan error.
- Koleksi Unik: Set hanya menyimpan elemen unik, tidak ada duplikasi.

2.26 Apa itu Dictionary?

Dictionary adalah struktur data yang menyimpan data dalam bentuk pasangan kunci (key) dan nilai (value). Mirip dengan kamus, di mana Anda mencari kata berdasarkan huruf pertama.

Key-Value Pair: Setiap item dalam dictionary terdiri dari kunci dan nilai yang terkait. Contoh: `{"1": "coffee"}`.

2.27 Keuntungan Menggunakan Dictionary

- Akses Cepat: Anda dapat mengakses nilai langsung menggunakan kunci, tanpa perlu mencari index seperti pada list. Ini membuatnya lebih cepat dan fleksibel.
- Mutable: Nilai dalam dictionary dapat diubah setelah dideklarasikan. Misalnya, Anda dapat mengubah nilai dari "tea" menjadi "mint tea".

2.28 Cara Mengakses dan Memanipulasi Dictionary

- Mengakses Nilai: Untuk mengakses nilai, gunakan sintaks berikut:

```
sample_dictionary = {1: "coffee", 2: "tea"}
print(sample_dictionary[1]) # Output: coffee
```

- Mengupdate Nilai: Untuk mengubah nilai, gunakan sintaks:

```
sample_dictionary[2] = "mint_tea"
```

- Menghapus Item: Untuk menghapus item, gunakan fungsi del:

```
del sample_dictionary[3] # Menghapus item dengan kunci
3
```

2.29 Iterasi Melalui Dictionary

Anda dapat menggunakan beberapa metode untuk iterasi, seperti:

- Standard Iteration: Menggunakan loop for untuk mengakses kunci.
- Items Function: Menggunakan `items()` untuk mendapatkan pasangan kunci-nilai.
- Values Function: Menggunakan `values()` untuk mendapatkan semua nilai.

2.30 Contoh Pembuatan dan Penggunaan Dictionary

- Deklarasi Dictionary:

```
my_d = {1: "test", "name": "Jim"}
```

- Mengakses Kunci:

```
print(my_d[1])    # Output: test
print(my_d["name"]) # Output: Jim
```

- Menambahkan Kunci Baru:

```
my_d[2] = "Test_2"
```

- Mengupdate Kunci:

```
my_d[1] = "not_a_test"
```

- Menghapus Kunci:

```
del my_d[1]
```

2.31 Apa itu args?

`args` adalah cara untuk mengirimkan sejumlah argumen non-keyword ke dalam fungsi. Sintaks:

```
def function_name(*args):
    # kode untuk memproses args
```

Contoh:

```
def sum_of(*args):
    total = 0
    for x in args:
        total += x
    return total
```

Jika kita memanggil `sum_of(4, 5, 6)`, hasilnya adalah 15.

2.32 Apa itu kwargs?

kwargs adalah cara untuk mengirimkan argumen keyword ke dalam fungsi, yang memungkinkan kita untuk menggunakan nama parameter. Sintaks:

```
def function_name(**kwargs):  
    # kode untuk memproses kwargs
```

Contoh:

```
def calculate_bill(**kwargs):  
    total = 0  
    for item, price in kwargs.items():  
        total += price  
    return round(total, 2)
```

2.33 Struktur data mana yang harus dipilih?

Bagian yang paling menantang bagi pengembang pemula adalah memahami struktur data mana yang paling sesuai dengan solusi yang dibutuhkan. Setiap struktur data menawarkan pendekatan yang berbeda dalam menyimpan, mengakses, dan memperbarui informasi yang disimpan di dalamnya. Ada banyak faktor yang perlu dipertimbangkan, termasuk ukuran, kecepatan, dan kinerja. Cara terbaik untuk memahami mana yang lebih sesuai adalah melalui contoh.

2.33.1 Contoh: Daftar Pegawai

Dalam contoh ini, terdapat daftar karyawan yang bekerja di sebuah restoran. Anda perlu dapat menemukan seorang karyawan berdasarkan ID karyawan mereka, ID numerik berbasis integer. Fungsi `get_employee` berisi loop for untuk mengiterasi daftar karyawan dan mengembalikan objek karyawan jika ID-nya cocok. Berikut adalah pemilihan struktur data yang salah

```
employee_list = [(12345, "John", "Kitchen"), (12458, "Paul",  
    "House_Floor")]  
  
def get_employee(id):  
    for employee in employee_list:  
        if employee[0] == id:  
            return {"id": employee[0], "name": employee[1],  
                "department": employee[2]}  
  
print(get_employee(12458))
```

Dalam kode ini, `employee_list` adalah daftar tuple. Kode ini berjalan dengan baik dan akan mengembalikan pengguna Paul, karena ID-nya, 12458, cocok. Tantangannya muncul ketika daftar menjadi lebih besar.

Alih-alih dua karyawan, Anda mungkin memiliki 2000 atau bahkan 20.000. Kode harus mengiterasi daftar secara berurutan hingga angka yang dicari cocok.

Anda dapat mengoptimalkan kode dengan membagi pencarian, tetapi bahkan dengan ini, kinerjanya masih kalah dibandingkan dengan struktur data lain, seperti kamus.

```
employee_dict = {
    12345: {
        "id": "12345",
        "name": "John",
        "department": "Kitchen"
    },
    12458: {
        "id": "12458",
        "name": "Paul",
        "department": "House_Floor"
    }
}

def get_employee_from_dict(id):
    return employee_dict[id];

print(get_employee_from_dict(12458));
```

Perhatikan bahwa, dalam blok kode ini, jika Anda mengubah struktur data menjadi kamus, Anda dapat menemukan karyawan tersebut. Perbedaan utamanya adalah Anda tidak perlu lagi mengiterasi daftar untuk menemukannya. Jika daftar membesar menjadi ukuran yang jauh lebih besar, waktu pencarian untuk menemukan karyawan tetap sama.

Ini adalah contoh yang baik tentang cara memilih struktur data yang tepat untuk solusi.

Keduanya bekerja dengan baik, tetapi pertukaran yang perlu dipertimbangkan adalah antara waktu dan skala. Solusi pertama akan bekerja dengan baik untuk jumlah data yang kecil, tetapi kinerjanya menurun seiring dengan pertumbuhan data.

Solusi kedua lebih cocok untuk jumlah data yang besar karena strukturnya memungkinkan waktu pencarian konstan, sehingga jumlah data yang besar dapat diakses dengan kecepatan konstan.

2.34 Errors

Error adalah kesalahan dalam kode yang biasanya disebabkan oleh kesalahan manusia, seperti kesalahan pengetikan.

Contoh:

- Syntax Error:

Kesalahan yang terjadi ketika kode tidak mengikuti aturan sintaksis Python. Misalnya, tidak menambahkan titik dua (:) di akhir kondisi.

```
if condition
    # kode
```

Kesalahan ini biasanya ditandai oleh IDE dan menunjukkan lokasi kesalahan.

- Indentation Error:

Kesalahan yang terjadi ketika indentasi tidak konsisten.

```
def function():
    print("Hello")
print("World") # Jika tidak diindentasikan dengan benar
```

Kesalahan ini menunjukkan bahwa indentasi tidak sesuai dengan yang diharapkan.

2.35 Exceptions

Exceptions adalah kesalahan yang terjadi saat eksekusi kode dan perlu ditangani oleh pengembang. Contoh:

- ZeroDivisionError: Terjadi ketika mencoba membagi angka dengan nol.

```
result = 5 / 0 # Ini akan menyebabkan ZeroDivisionError
```

Kesalahan ini menunjukkan bahwa operasi matematika tidak valid.

2.36 Penanganan Exceptions

Pengembang harus menangani exceptions untuk mencegah aplikasi gagal.

Contoh: Menggunakan blok `try` dan `except` untuk menangani exceptions.

```
try:
    result = 5 / 0
except ZeroDivisionError:
    print("Tidak bisa dibagi dengan nol.")
```

Blok `try` mencoba menjalankan kode, dan jika terjadi exception, blok `except` akan menangani kesalahan tersebut.

2.37 Penanganan Exceptions

- `try`: Blok kode yang akan dieksekusi. Jika terjadi kesalahan, eksekusi akan berpindah ke blok `except`.
- `except`: Blok kode yang akan dijalankan jika terjadi exception. Ini memungkinkan kita untuk memberikan pesan kesalahan yang lebih ramah pengguna.

2.38 Mengakses Informasi Exception

- `as e`: Menggunakan alias untuk exception yang terjadi, sehingga kita bisa mencetak informasi lebih lanjut.
- `e.__class__`: Mengakses kelas dari exception yang terjadi untuk memberikan informasi lebih spesifik.

2.39 Menangani Beberapa Exceptions

Chaining except: Kita bisa menambahkan beberapa pernyataan except untuk menangani berbagai jenis exceptions.

```
try:
    result = divide_by(40, 0)
except ZeroDivisionError as e:
    print("Division by zero:", e)
except Exception as e:
    print("An error occurred:", e)
```

2.40 File Handling: open dan close

- `open`: Digunakan untuk membuka, membaca, dan menulis file. Memerlukan dua argumen: nama file dan mode.
 - `r`: Membuka file untuk dibaca dalam format teks (Default).
 - `rb`: Membuka file untuk dibaca dalam format biner.
 - `r+`: Membuka file untuk membaca dan menulis.
 - `w`: Membuka file untuk menulis (akan menimpa file yang ada).
 - `a`: Membuka file untuk menambahkan data.
- `close`: Digunakan untuk menutup koneksi file yang terbuka. Tidak memerlukan argumen.
- Membuka file untuk membaca:

```
file = open("test.txt", "r")
data = file.readline()
print(data)
file.close()
```

2.41 With Statement

Pernyataan `with open` adalah cara yang lebih baik untuk membuka file karena secara otomatis menutup file setelah selesai digunakan. Ini juga lebih baik dalam menangani kesalahan.

```
with open("test.txt", "r") as file:
    data = file.readline()
    print(data)
```

2.42 Membuat File

Fungsi `open()` digunakan untuk membuat file baru. Sintaksnya:

```
with open("newfile.txt", "w") as file:
```

`open()` membuka file dengan mode tertentu. Mode "w" berarti write (tuliskan), yang akan membuat file baru atau menimpa file yang sudah ada.

2.43 Menyisipkan Konten

- `write()`: Metode ini digunakan untuk menulis teks ke dalam file. Sintaksnya:

```
file.write("This is a new file created.")
```

`write()` menambahkan string ke file yang dibuka.

- `writelines()`: Metode ini digunakan untuk menulis beberapa baris ke dalam file. Sintaksnya:

```
file.writelines(["This is the first line.\n", "This is the second line."])
```

`writelines()` menerima daftar string dan menulisnya ke file. Untuk memisahkan baris, gunakan `\n` untuk membuat baris baru.

2.44 Mengganti dan Menambahkan Konten

Mode Append untuk menambahkan konten tanpa menimpa file yang ada, gunakan mode "a". Sintaksnya:

```
with open("newfile.txt", "a") as file:
```

Mode "a" (append) memungkinkan penambahan konten ke akhir file yang sudah ada.

2.45 Menangani Kesalahan (Exception)

`try` dan `except` digunakan untuk menangani kesalahan saat membuat file. Sintaksnya:

```
try:
    with open("sample/newfile.txt", "w") as file:
except FileNotFoundError as e:
    print(f"Error: {e}")
```

2.46 Metode untuk Membaca File

- **read:**

Mengembalikan seluruh isi file sebagai string. Anda juga dapat memberikan argumen integer untuk mengembalikan jumlah karakter tertentu.

```
file.read(size)
```

Mengambil semua karakter dari file atau sejumlah karakter yang ditentukan.

- **readline:**

Mengembalikan satu baris dari file sebagai string. Anda juga dapat memberikan argumen integer untuk mengembalikan sejumlah karakter dari baris tersebut.

```
file.readline(size)
```

Mengambil satu baris dari file, atau sejumlah karakter dari baris pertama jika argumen diberikan.

- **readlines:**

Membaca seluruh isi file dan mengembalikannya dalam bentuk daftar yang terurut.

```
file.readlines()
```

Mengambil semua baris dari file dan mengembalikannya sebagai daftar, memungkinkan iterasi atau pemilihan baris tertentu.

2.47 Jalur File

- **Absolute Path:**

Mengandung informasi lengkap untuk menemukan file, termasuk label drive atau tanda "/" di depan.

Definisi: Jalur yang menunjukkan lokasi file secara lengkap, terlepas dari direktori saat ini.

- Relative Path:

Tidak mengandung referensi ke direktori root dan biasanya relatif terhadap file yang memanggil. Definisi: Jalur yang hanya mencakup informasi yang diperlukan untuk menemukan file dalam direktori kerja saat ini.

2.48 Contoh Penggunaan Open Statement

- Membuka file dengan mode baca:

```
with open('sample.txt', 'r') as file:
    content = file.read()
    print(content)
```

Menggunakan `with open` untuk membuka file, yang secara otomatis menutup file setelah selesai digunakan.

- Menggunakan `read` untuk mencetak isi file:

```
print(file.read())
```

Mencetak seluruh isi file.

- Menggunakan `readline` untuk mencetak baris pertama:

```
print(file.readline())
Mencetak hanya baris pertama dari file.
```

- Menggunakan `readlines` untuk mendapatkan daftar baris:

```
lines = file.readlines()
for line in lines:
    print(line)
```

Mengambil semua baris sebagai daftar dan mencetaknya satu per satu.

3 Paradigma Pemrograman

4 Modul, Paket, Pustaka, and Alat-alat